

TD&TP 9 : AB & PROGRAMMATION GÉNÉRIQUE

Exercice 1

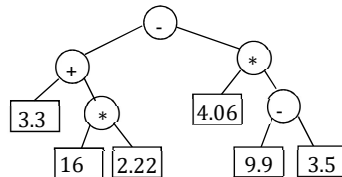
On cherche à évaluer une expression arithmétique complètement parenthésée, constituée d'opérandes, d'opérateurs binaires ('+', '-', '*', '/'), de parenthèses ('(', ')') et d'espaces comme séparateur.

Pour ce faire, on transforme l'expression sous forme d'arbre binaire puis évalue l'arbre ainsi obtenu en utilisant un parcours post-fixé.

On suppose que les opérandes sont des nombres réels.

Soit l'expression parenthésée $\text{Exp} = ((3.3 + (16 * 2.22)) - (4.06 * (9.9 - 3.5)))$

Sa représentation en arbre binaire donne l'arbre binaire ci-dessous :



- 1) Transformer l'expression complètement parenthésée donnée en une expression post-fixée équivalente représentée par un arbre.
- 2) Evaluer l'expression post-fixée.
- 3) Détecter les erreurs possibles.

Exercice 2

Implémenter l'exercice 1 d'une manière générique.

Annexe : Généricité

En programmation, la **généricité** (ou **programmation générique**), consiste à définir des algorithmes identiques opérant sur des données de types différents.

On souhaite que les structures de données (que ce soit des tables, des listes, des arbres, etc) puissent être utilisées quel que soit le type des données qui y sont rangés. C'est la notion de structures de données et de fonctions *génériques*.

Les fonctions créées pour manipuler les structures de données doit fonctionner **quel que soit le type de données stockées**. Dans la plupart des langages modernes, ce genre de problématique a été étudié et des mécanismes sont prévus spécialement pour ça : les templates en C++, les génériques en Java, les modules paramétrés en OCaml, etc.

Malheureusement, en C, ce n'est pas aussi simple que ça. Il existe des mécanismes permettant de mettre en œuvre une forme de programmation générique, mais cela demande bien plus de rigueur et de compréhension du bas niveau que les templates de C++.

Exemple

Reprenons l'exemple du parcours en largeur d'un arbre binaire à l'aide d'une file. Plutôt que d'utiliser des files dont le type des données est `<< Arbre >>`, c'est-à-dire d'écrire dans le fichier définissant les opérations sur les Files

```
typedef Arbre TElement_Arbre ;
```

Ce qui devrait être modifié si on changeait le type des cellules, il est préférable de définir

```
typedef void *TElement_Arbre ;
```

La fonction de parcours en largeur n'a pas besoin d'être modifiée.

Pointeur générique void* (où Pointeur sur n'importe quoi)

Le langage C est un langage **typé statiquement**, ce qui signifie que le type de toutes les variables doit être connu au moment de la compilation. C'est vrai pour les variables, mais également pour les pointeurs : bien que tous les pointeurs désignent une adresse mémoire, un pointeur sur entier (`int*`) sera différent d'un pointeur sur un nombre à virgule flottante (`float*`), puisque le type du pointeur définit le type de la variable sur laquelle le pointeur pointe.

Ainsi, le bout de code suivant produira un warning.

```
int i = 2;
float* p = &i; //< Initialization from incompatible pointer type
```

Le pointeur générique **void*** échappe à cette règle.

Lorsque l'on déclare un (void*), on peut y stocker l'adresse de n'importe quoi. La contrepartie, c'est que comme le compilateur n'a pas la moindre idée du type de la donnée pointée, il est **impossible de déréférencer un (void*) sans le caster**.

Il faut faire attention car ce n'est pas un pointeur vers un objet de type void mais bien un type à part entière. D'ailleurs au passage, un pointeur vers un type **void** n'a pas de sens puisque le type **void** n'est pas un type qui doit contenir quelque chose, il est là pour indiquer une absence de type.

<pre>typedef struct cellule{ void *donnee ; struct cellule *suivant ; }*Liste;</pre>	<pre>typedef struct noeud void *donnee ; struct noeud *filsG ; struct noeud *filsD ; }*Arbre;</pre>
--	---

Le membre donné de la structure est un pointeur générique, pointant vers un objet dont on n'a pas spécifié le type.

Premiers pas avec le pointeur générique

Nous avons déjà rencontré le pointeur générique, notamment lors d'allocations mémoire. En effet, la fonction **malloc** a pour signature : **void * malloc(size_t size)**

La fonction renvoie un pointeur générique. Ceci est tout à fait normal puisque la fonction ne sait pas quel doit être le type de retour, ce qui lui importe, c'est la taille à allouer. Ne connaissant pas la taille du type à allouer, elle doit donc renvoyer un pointeur vers n'importe quel type d'objet, il s'agit donc du pointeur générique. Comme le pointeur est générique, il doit pouvoir être affecté sans problème. De plus, on doit pouvoir lui affecter un pointeur sur un type donné.

L'exemple suivant montre l'utilisation possible d'un pointeur générique pour afficher l'adresse d'une variable :

```
#include <stdio.h>
#include <stdlib.h>
void main ( void ){
    void * ptr = NULL;
    int a = 2;
    ptr = &a;
    printf("%p\n",ptr);
} //Ici, le pointeur générique remplace donc un pointeur sur un entier.
```

Usages de void *

L'usage le plus fréquent de ce type de pointeurs se rencontre dans une déclaration des arguments d'une fonction.

On souhaite créer une fonction capable de prendre en compte différents types d'arguments : int, float, personne, vache, etc.

En déclarant le pointeur vers cet argument comme étant de type void *, on peut utiliser types plusieurs d'arguments.

Exemple permutation(void *a, void *b)

Mais avoir passé un pointeur est une chose, pouvoir récupérer correctement la valeur vers laquelle il pointe en est une autre chose.

Il faut pour cela **caster** ce pointeur afin que le compilateur sache à quoi s'en tenir et fasse son travail correctement.

Exemple 1 :

```
// la fonction moitié calcule la moitié de la valeur qui lui est passée.
#include <stdio.h>
typedef struct {
    double pr;
    double pi;
}Complexe;

void moitié(void *x, char t){
    //Caster la valeur de x selon le type de x est faite la division par 2
    switch(t){
        case 'i' : *(int*)x /=2; break;
        case 'd' : *(double*)x /=2.0;break;
        case 'f' : *(float*)x /=2.0;break;
        case 'c' : { Complexe * y;
                    y = (Complexe*)x;
                    y->pr /=2.0;
                    y->pi /=2.0;
                    break;
                }
    }
}

void main(void){
    int n=4;
    float f=8.0;
    double d=10.0;
    Complexe c={6, 6};
    moitié(&n, 'i');
    printf("\n n=%d", n);
    moitié(&f, 'f');
    printf("\n f=%f", f);
    moitié(&d, 'd');
    printf("\n d=%lf", d);
    moitié(&c, 'c');
    printf("\n c=%lf %lf", c.pr, c.pi);
    printf("\n");
}
```

Les limites du pointeur générique

Soi la fonction permutation qui échange deux éléments quels qu'en soient leurs types (il faut au moins que les deux éléments soient du même type).

A première vue, nous pourrions penser à une fonction utilisant des pointeurs génériques comme ceci :

```
void permutation( void * a , void * b){
    void * tmp;

    tmp = a;
    a = b;
    b = tmp;
}

void main(void) {

    int a = 3, b=2;

    permutation( &a, &b );
    printf( "a = %d , b = %d" , a , b );

}
```

Le résultat est a=3, b=2

Explication : la fonction permutation ici a transmis une copie de ses arguments et non leurs adresses. Ce ne sont pas les variables a et b que nous échangeons mais leur copie.

La fonction qui effectue l'échange de deux entiers est :

```
void permutation_int( int * a, int * b){
    int tmp;

    tmp = *a;
    *a = *b;
    *b = tmp;
}
```

Exemple 2

//Permutation de deux variables de n'importe quel type.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>      /* memcpy() */
typedef struct{
    double pr, pi;
} Complexe;

void permutation( void * a , void * b, unsigned int taille){
    void * tmp;

    tmp = malloc(taille);
    memcpy(tmp, a, taille);    /* tmp <- a */
    memcpy(a, b, taille);     /* a <- b */
    memcpy(b, tmp, taille);    /* b <- tmp */
}

void affComplexe(Complexe c){
    printf("\nC=%lf %lf",c.pr, c.pi);
}

int main ( void ){
    int a = 3, b = 2;
    char c='c', d='d';
    double x=10.0, y=20.0;
    Complexe c1={2, 2}, c2={7, 7};

    permutation( &a, &b, sizeof(int) );
    printf( "a = %d , b = %d\n" , a , b );

    permutation( &c, &d, sizeof(char) );
    printf( "c = %c , d = %c\n" , c , d );

    permutation( &x, &y, sizeof(double) );
    printf( "x = %lf , y = %lf\n" , x , y );

    affComplexe(c1);
    affComplexe(c2);
    permutation( &c1, &c2, sizeof(Complexe) );
    affComplexe(c1);
    affComplexe(c2);
}
```